System Software

Process Management

A simple, step-by-step study guide

Processes | States | Scheduling | IPC | Threads | Deadlocks

Table of Contents

1. Introduction to System Software 3

2. What is a Process? 5

3. Process States and Life Cycle 7

4. Process Control Block (PCB) 9

5. Process Creation and Termination 10

6. Process Scheduling: Queues and Schedulers 12

7. CPU Scheduling Algorithms 13

8. Context Switching 16

9. Inter-Process Communication (IPC) 17

10. Threads 18

11. Process Synchronization 20

12. Deadlock 22

13. Summary, Practice Questions and Glossary 24

How to use this guide
Each chapter starts with the simple idea, then explains the details, and ends with a quick summary. Look for the **Key Idea** boxes (blue) and **Real-life Analogy** boxes (yellow) - they make hard concepts easy to remember.

CHAPTER 1

Introduction to System Software

Every computer has two kinds of programs. **Application software** helps you do tasks such as writing a letter, browsing the web or playing a game. **System software** works quietly in the background and makes the computer usable in the first place. It manages the hardware, provides services to applications and keeps everything running smoothly.

1.1 Types of system software

Type	What it does	Examples
Operating System	Manages CPU, memory, files and devices; runs programs	Windows, Linux, Android, macOS
Device Drivers	Let the OS talk to hardware such as printers and keyboards	Graphics driver, printer driver
Compilers / Interpreters	Translate high-level code into machine code	GCC, Python interpreter, javac
Assembler	Converts assembly language to machine language	NASM, MASM
Linker and Loader	Joins program pieces and loads them into memory	ld, dynamic loader
Utility Programs	Maintenance tasks like backup and antivirus	Disk cleanup, file manager

1.2 Why is the Operating System the heart of system software?

The operating system (OS) is the most important piece of system software. It sits between the user (and applications) and the hardware. It has five major jobs: **process management**, **memory management**, **file management**, **device management** and **security**. This guide focuses on the first job - process management - because it decides how the CPU is shared between all running programs.

Real-life Analogy
Think of the OS as the manager of a busy restaurant. Customers (programs) want food, the kitchen (CPU) can only cook so many dishes at once, and the manager decides who is served next, keeps orders organised, and makes sure nobody is left waiting forever.

1.3 What you will learn

- What a program and a process are, and how they differ
- The life cycle of a process and the information the OS keeps about it
- How the OS creates, schedules, switches between and ends processes
- How processes talk to each other (IPC) and how threads work
- How the OS keeps shared data safe (synchronization) and avoids deadlocks

Key Idea
Process management means the OS decides which program gets the CPU, for how long, and keeps all the programs from interfering with each other.

CHAPTER 2

What is a Process?

2.1 Program vs Process

A **program** is just a file stored on disk. It contains instructions but does nothing by itself - it is *passive*. When you double-click it, the OS loads it into memory and starts executing it. That running instance is called a **process** - it is *active*.

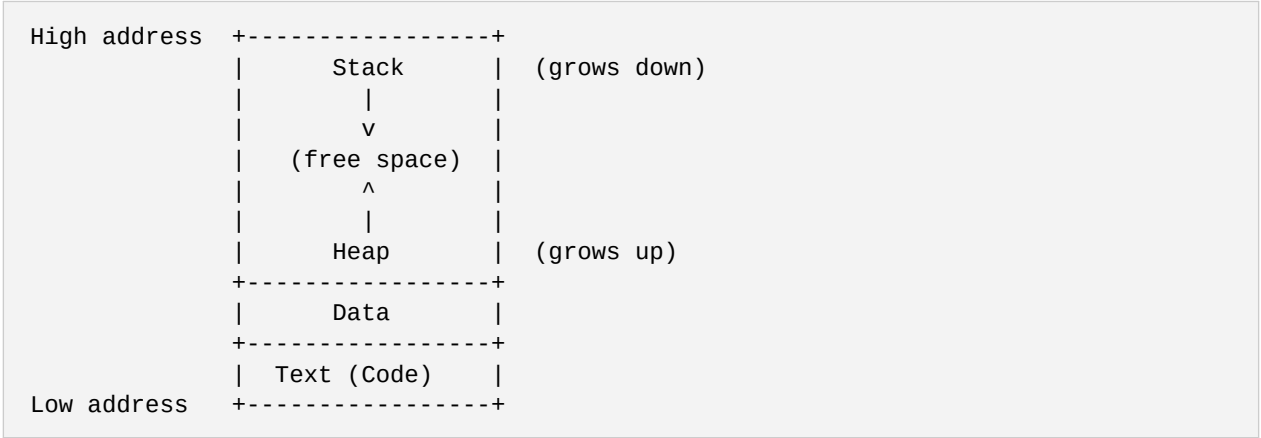
Program	Process
Passive entity (a file on disk)	Active entity (running in memory)
Exists as long as the file is not deleted	Exists only while it is executing
Needs only disk space	Needs CPU, memory, files and I/O devices
One program = one file	One program can create many processes

Real-life Analogy
A recipe in a cookbook is a program. A chef actually cooking that recipe in the kitchen is a process. Two chefs cooking the same recipe are two separate processes of one program.

2.2 Parts of a process in memory

When a process is loaded, the OS gives it a private area of memory divided into four parts:

Section	Contents
Text (Code)	The program instructions to be executed. Read-only.
Data	Global and static variables that exist for the whole run.
Heap	Memory created dynamically while running (e.g. malloc, new). Grows upward.
Stack	Function parameters, local variables and return addresses. Grows downward.



2.3 Why do we need processes?

Modern computers do many things at once: you listen to music, download a file and type a document at the same time. The CPU can run only one instruction stream at a time (per core), so the OS switches between processes so quickly that it looks like they run together. This is called **multiprogramming** (many programs in memory) and **multitasking** (quickly switching between them).

> **Key Idea**
> A process = a program in execution. It has its own code, data, heap, stack and a set of CPU register values.

CHAPTER 3

Process States and Life Cycle

From the moment it is created until it finishes, a process moves through different **states**. The OS tracks the state of every process.

3.1 The five-state model

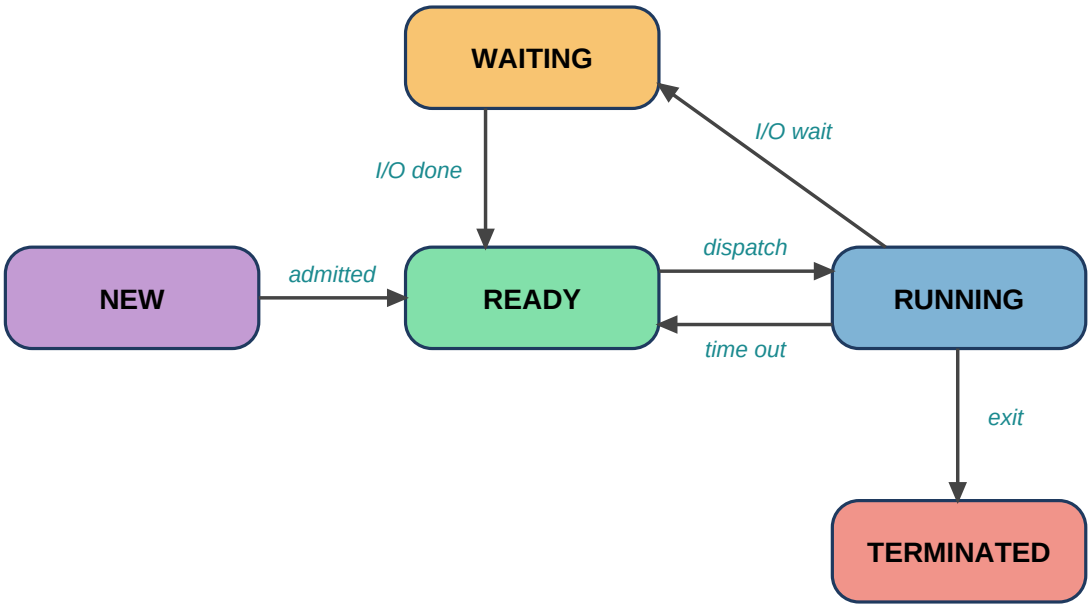


Figure 3.1 - Process state diagram. Arrows show how a process moves from one state to another.

State	Meaning
New	The process is being created. The OS is setting up its resources.
Ready	The process is in memory and waiting for its turn on the CPU.
Running	The CPU is currently executing the process's instructions.
Waiting (Blocked)	The process cannot continue until some event happens, e.g. disk read finishes or the user presses a key.
Terminated	The process has finished. The OS cleans up its resources.

3.2 State transitions explained

Transition	When it happens
New -> Ready	The OS admits the process into the ready queue.
Ready -> Running	The scheduler picks the process and gives it the CPU (dispatch).
Running -> Ready	The time slice ends or a higher-priority process arrives (pre-emption).
Running -> Waiting	The process requests I/O or waits for an event.
Waiting -> Ready	The I/O or event completes; the process is ready again.

Transition	When it happens
Running -> Terminated	The process finishes or is killed.

Real-life Analogy
At a bank: you walk in and take a token (New), sit in the waiting area (Ready), go to the counter when called (Running), get sent to fetch a missing document (Waiting) and, once done, leave the bank (Terminated).

Key Idea
Only one process per CPU core can be in the Running state at a time. Many processes can be Ready or Waiting.

CHAPTER 4

Process Control Block (PCB)

How does the OS remember everything about a process, especially when it must pause one and run another? It keeps a data structure called the **Process Control Block (PCB)**, also called a *task control block*. Every process has exactly one PCB, and it is created when the process is created and deleted when the process ends.

4.1 What a PCB contains

Field	Purpose
Process ID (PID)	A unique number that identifies the process.
Process state	New, Ready, Running, Waiting or Terminated.
Program Counter	Address of the next instruction to execute.
CPU registers	Saved values of all registers (accumulator, stack pointer, etc.).
Scheduling information	Priority, pointers to scheduling queues, time used.
Memory information	Base/limit registers, page tables, segment tables.
Accounting information	CPU time used, time limits, job numbers.
I/O status information	List of open files and I/O devices allocated.

4.2 Why the PCB is important

- **Saves progress:** when a process is paused, its register values and program counter are stored in the PCB so it can resume exactly where it left off.
- **Tracks resources:** the OS knows which files, memory and devices belong to which process.
- **Helps scheduling:** priority and state fields let the scheduler choose the next process.
- **Enables protection:** the OS can stop a process from touching another process's memory.

Real-life Analogy
A PCB is like a student's file in the school office - name, roll number, class, marks, attendance and fees all in one folder. Whenever the school needs to know something about the student, it opens that file.

4.3 Process table

The OS keeps all PCBs together in a **process table**. When you run the command `ps` in Linux or open Task Manager in Windows, you are viewing information drawn from this table.

Key Idea
PCB = the identity card and progress report of a process. Without it, the OS could not pause and resume processes.

CHAPTER 5

Process Creation and Termination

5.1 How processes are created

A new process is always created by an existing process. The creator is called the **parent** and the new process is the **child**. Children can create their own children, forming a **process tree**. When the computer boots, the OS creates a first process (called *init* or *systemd* in Linux) and every other process descends from it.

```
init (PID 1)
 |-- login shell (PID 210)
 |     |-- text editor  (PID 355)
 |     +-- compiler     (PID 361)
 +-- network service    (PID 230)
```

Common events that cause process creation:

- System start-up (background services start).
- A user opens an application or runs a command.
- A running process asks the OS to create another (using a system call).
- A batch job is started.

5.2 Parent and child relationship

Question	Possible choices
Resource sharing	Child shares all, some or none of the parent's resources.
Execution	Parent and child run concurrently, or the parent waits until the child finishes.
Address space	Child is a duplicate of the parent, or the child loads a brand-new program.

5.3 fork(), exec() and wait() in UNIX/Linux

- `fork()` - creates a child process that is a copy of the parent. It returns 0 in the child and the child's PID in the parent.
- `exec()` - replaces the process's memory with a new program.
- `wait()` - makes the parent pause until the child finishes.

```
pid = fork();
if (pid == 0) {
    execlp("/bin/ls", "ls", NULL);   /* child runs ls */
} else {
    wait(NULL);                      /* parent waits  */
    printf("Child finished\n");
}
```

5.4 Process termination

A process ends when it executes its last statement and calls `exit()`. The OS then frees its memory, closes its files and returns a status value to the parent. A parent may also terminate a child if the child uses too many resources, its task is no longer needed, or the parent itself is exiting.

Special case	Meaning
Zombie process	Child has finished, but the parent has not yet collected its exit status. Only its PCB entry remains.
Orphan process	Parent finished before the child. The init process adopts the child.

Key Idea
Processes form a family tree: parents create children using fork(); children may run new programs using exec().

CHAPTER 6

Process Scheduling: Queues and Schedulers

With many processes and only one (or a few) CPUs, the OS needs a fair and efficient way to decide who runs next. This decision-making is called **process scheduling**. The main goal is to keep the CPU busy all the time and give users quick responses.

6.1 Scheduling queues

Queue	Holds
Job queue	All processes in the system.
Ready queue	Processes in main memory that are ready and waiting for the CPU.
Device (I/O) queue	Processes waiting for a particular I/O device such as a disk or printer.

A process keeps moving between these queues during its life. For example, it waits in the ready queue, gets the CPU, asks for disk data, moves to the disk's device queue, and later returns to the ready queue.

6.2 Types of schedulers

Scheduler	Also called	Role	Runs
Long-term	Job scheduler	Chooses which jobs are brought into memory; controls how many processes run at once (degree of multiprogramming).	Rarely
Short-term	CPU scheduler	Chooses which ready process gets the CPU next.	Very often (milliseconds)
Medium-term	Swapper	Temporarily removes (swaps out) processes from memory and brings them back later to reduce load.	Occasionally

6.3 CPU-bound and I/O-bound processes

I/O-bound processes spend most of their time waiting for input or output (e.g. a web browser). **CPU-bound** processes spend most of their time computing (e.g. video rendering). A good long-term scheduler mixes both types so that the CPU and I/O devices are both kept busy.

6.4 Dispatcher

After the short-term scheduler chooses a process, the **dispatcher** actually hands over the CPU. It switches context, switches to user mode and jumps to the correct location in the program. The time the dispatcher takes is called **dispatch latency**, and it should be as small as possible.

> **Key Idea**
> Scheduler = decides WHO runs next. Dispatcher = actually GIVES the CPU to that process.

CHAPTER 7

CPU Scheduling Algorithms

7.1 Important terms

Term	Meaning
Arrival time (AT)	Time at which the process enters the ready queue.
Burst time (BT)	Total CPU time the process needs.
Completion time (CT)	Time at which the process finishes.
Turnaround time (TAT)	CT - AT (total time from arrival to completion).
Waiting time (WT)	TAT - BT (time spent waiting in the ready queue).
Response time	Time from arrival until the process first gets the CPU.

7.2 Preemptive vs non-preemptive

In **non-preemptive** scheduling, once a process gets the CPU it keeps it until it finishes or waits for I/O. In **preemptive** scheduling, the OS may take the CPU away from a running process, for example when its time slice ends or a more important process arrives.

7.3 Goals of a good scheduler

- Maximise CPU utilisation and throughput (jobs completed per unit time).
- Minimise turnaround time, waiting time and response time.
- Be fair - no process should wait forever (avoid **starvation**).

Example used in this chapter. Four processes all arrive at time 0:

Process	Burst time (ms)
P1	6
P2	8
P3	7
P4	3

7.4 First-Come, First-Served (FCFS)

The simplest algorithm: the process that arrives first is served first, like a queue at a ticket counter. It is non-preemptive.

FCFS Gantt chart

Process	BT	CT	TAT	WT
P1	6	6	6	0
P2	8	14	14	6
P3	7	21	21	14
P4	3	24	24	21
Average			**16.25**	**10.25**

Drawback: the *convoy effect* - one long process makes all short processes wait behind it.

7.5 Shortest Job First (SJF)

The process with the smallest burst time runs first. It gives the lowest average waiting time but needs to know burst times in advance. The preemptive version is called **Shortest Remaining Time First (SRTF)**.

SJF Gantt chart

0 3 9 16 24

Process	BT	CT	TAT	WT
P4	3	3	3	0
P1	6	9	9	3
P3	7	16	16	9
P2	8	24	24	16
Average			**13.00**	**7.00**

Drawback: long processes may starve if short ones keep arriving.

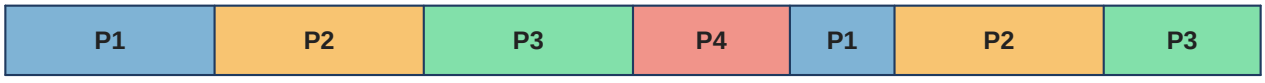
7.6 Priority Scheduling

Each process is given a priority number, and the CPU goes to the process with the highest priority (in many systems a smaller number means higher priority). It can be preemptive or non-preemptive. Low-priority processes may starve, which is solved using **aging** - slowly raising the priority of a process the longer it waits.

7.7 Round Robin (RR)

Designed for time-sharing systems. Each process gets a small fixed slice of CPU time called a **time quantum**. If it is not finished when the quantum ends, it goes to the back of the ready queue. It is preemptive and fair.

Round Robin Gantt chart (quantum = 4 ms)



0 4 8 12 15 17 21 24

Process	BT	CT	TAT	WT
P1	6	17	17	11
P2	8	21	21	13
P3	7	24	24	17
P4	3	15	15	12
Average			**19.25**	**13.25**

Choosing the quantum: if it is too large, RR behaves like FCFS; if it is too small, the CPU wastes time on too many context switches. A common rule is that about 80% of CPU bursts should be shorter than the quantum.

7.8 Multilevel Queue and Multilevel Feedback Queue

The ready queue is divided into several queues, for example *system processes*, *interactive processes* and *batch processes*, each with its own algorithm. In a **multilevel feedback queue**, a process can move between queues: a process that uses too much CPU is moved to a lower-priority queue, and one that has waited long is moved up. This is flexible and is used in many real operating systems.

7.9 Comparison

Algorithm	Type	Strength	Weakness
FCFS	Non-preemptive	Simple, fair order	Convoy effect, long waits
SJF / SRTF	Both	Lowest average waiting time	Starvation; needs burst estimates
Priority	Both	Important jobs first	Starvation (fix: aging)
Round Robin	Preemptive	Good response time, fair	Overhead if quantum is small
Multilevel Feedback	Preemptive	Flexible and adaptive	Complex to tune

Key Idea
In our example, SJF gave the best average waiting time (7.00 ms), FCFS 10.25 ms and RR 13.25 ms - but RR gives every process a quick first response, which matters for interactive use.

CHAPTER 8

Context Switching

When the CPU moves from running one process to running another, the OS performs a **context switch**. The *context* of a process is the information in its PCB - the program counter, registers and memory details.

8.1 Steps in a context switch

1. An interrupt or system call occurs (for example, the timer signals that the time slice is over).
2. The OS saves the state (registers, program counter) of the running process into its PCB.
3. The scheduler selects the next process from the ready queue.
4. The OS loads the saved state of the chosen process from its PCB.
5. The CPU resumes executing the new process from exactly where it last stopped.

8.2 Context-switch overhead

During a switch the CPU does no useful work for user programs - this time is pure **overhead**. It typically takes a few microseconds, but depends on the hardware, the number of registers to save and memory management work. Doing too many context switches slows the whole system.

Real-life Analogy
Imagine reading two books alternately. Each time you switch, you put a bookmark in the first book (save state), open the second at its bookmark (load state) and continue. The time spent bookmarking is the overhead.

8.3 Why context switching is useful

- Allows multitasking - several programs appear to run at once.
- Keeps the CPU busy when one process is waiting for I/O.
- Lets the OS respond quickly to urgent events and interrupts.

8.4 Interrupts

An **interrupt** is a signal telling the CPU to pause what it is doing and attend to something urgent, such as a keystroke, a timer tick or a finished disk transfer. The CPU saves its current state, runs a small program called an **interrupt handler**, then continues. Interrupts are what trigger most context switches.

Key Idea
Context switch = save the old process's state + load the new process's state. It is necessary but costs time.

CHAPTER 9

Inter-Process Communication (IPC)

Processes run in separate memory spaces so they cannot accidentally damage each other. But sometimes they must cooperate - for example, a web server passes data to a database. The OS provides **Inter-Process Communication (IPC)** mechanisms for this.

9.1 Independent vs cooperating processes

An **independent** process cannot affect or be affected by other processes. A **cooperating** process can. We need cooperation for information sharing, faster computation (split a job between processes), modularity and convenience.

9.2 Two basic IPC models

	Shared Memory	Message Passing
How it works	Processes share a region of memory and read/write it directly	Processes exchange messages using send() and receive()
Speed	Faster (system calls only needed to set up)	Slower (kernel involved in each message)
Best for	Large amounts of data on the same machine	Small data; works across networks too
Challenge	Must synchronise access to avoid conflicts	Message size and delivery handling

9.3 Common IPC techniques

- **Pipes:** a one-way channel where one process writes and another reads (like `ls | grep txt`).
- **Named pipes (FIFO):** pipes with a name in the file system so unrelated processes can use them.
- **Message queues:** the OS stores messages until the receiver is ready.
- **Shared memory:** the fastest method; needs semaphores or mutexes for safety.
- **Sockets:** communication between processes on different computers over a network.
- **Signals:** short notifications such as 'stop' or 'terminate' sent to a process.

9.4 Producer-Consumer problem

A classic example of cooperation: a **producer** creates items and puts them into a **buffer**; a **consumer** takes items out. The producer must wait when the buffer is full and the consumer must wait when it is empty.

Real-life Analogy
A baker (producer) places bread on a shelf (buffer) and a customer (consumer) takes it. If the shelf is full the baker must stop; if it is empty the customer must wait.

Key Idea
IPC lets separate processes cooperate. Shared memory is fast; message passing is simpler and safer.

CHAPTER 10

Threads

A **thread** is the smallest unit of CPU execution - often called a *lightweight process*. A process can contain one or many threads. Threads of the same process share code, data and open files, but each has its own program counter, registers and stack.

10.1 Process vs thread

Process	Thread
Heavyweight; has its own memory space	Lightweight; shares memory with other threads in the process
Creation and switching are slow	Creation and switching are fast
Communication needs IPC	Communicate directly through shared data
If one process crashes, others are usually safe	If one thread crashes, the whole process may crash

Real-life Analogy
A process is a company; threads are its employees. Employees share the office, files and equipment (shared resources) but each works on their own task with their own desk and notes (own stack and registers).

10.2 Benefits of multithreading

- **Responsiveness:** a program stays usable even if one part is busy (a browser loads a page while you scroll).
- **Resource sharing:** threads share memory by default.
- **Economy:** cheaper to create and switch than processes.
- **Scalability:** threads can run in parallel on multiple CPU cores.

10.3 Types of threads

Type	Managed by	Pros / Cons
User-level threads	A thread library in user space	Fast to create and switch; but if one thread blocks, the whole process may block.
Kernel-level threads	The operating system	Can run on multiple cores and block independently; but slower to manage.

10.4 Multithreading models

- **Many-to-One:** many user threads map to one kernel thread (no true parallelism).
- **One-to-One:** each user thread has its own kernel thread (used by Linux and Windows).
- **Many-to-Many:** many user threads map to a smaller or equal number of kernel threads.

Key Idea
Threads let one process do many things at once while sharing the same memory - faster and lighter than creating many processes.

CHAPTER 11

Process Synchronization

When several processes or threads share data, the final result can depend on who runs first. If two processes change the same variable at the same time, the result can be wrong. This is called a **race condition**.

11.1 A race condition example

Suppose two processes both add 1 to a shared variable `count` that starts at 5. Each does: (1) read count, (2) add 1, (3) write back. If both read 5 before either writes, both write 6 - and one update is lost. The correct answer should be 7.

```
Process A: read count (5)
Process B: read count (5)
Process A: write count = 6
Process B: write count = 6    <-- lost update! should be 7
```

11.2 The Critical Section problem

The part of a program that accesses shared data is the **critical section**. A good solution must satisfy three rules:

Requirement	Meaning
Mutual exclusion	Only one process may be inside its critical section at a time.
Progress	If no one is in the critical section, a waiting process must be able to enter without unnecessary delay.
Bounded waiting	There is a limit on how long a process must wait to enter; nobody waits forever.

```
do {
    entry section      /* ask permission */
    critical section   /* use shared data */
    exit section       /* release permission */
    remainder section  /* other work */
} while (true);
```

Real-life Analogy
A single-occupancy restroom has a lock. Only one person can be inside (mutual exclusion); when it is free, the next person enters (progress); and people queue in order so nobody waits forever (bounded waiting).

11.3 Tools for synchronization

Mutex lock

A mutex has two operations: **acquire()** (lock) and **release()** (unlock). A process must acquire the lock before entering the critical section and release it afterwards.

Semaphore

A **semaphore** is an integer variable accessed only through two atomic operations: **wait()** (also P) decreases the value and blocks if it would go below zero; **signal()** (also V) increases the value and wakes a waiting process.

```
wait(S):   while (S <= 0) ;  /* wait */
           S = S - 1;
signal(S): S = S + 1;
```

- **Binary semaphore** (0 or 1) works like a mutex.
- **Counting semaphore** controls access to a pool of identical resources, e.g. 3 printers.

Monitor

A **monitor** is a high-level construct that bundles shared data and the procedures that use it, and allows only one process inside at a time. Many programming languages such as Java provide it (via the `synchronized` keyword).

> **Key Idea**
> Synchronization makes sure shared data stays correct. The main tools are mutex locks, semaphores and monitors.

CHAPTER 12

Deadlock

A **deadlock** occurs when a group of processes are all waiting for resources held by one another, so none of them can ever continue.

> **Real-life Analogy**
> Two cars meet on a narrow bridge from opposite sides. Neither can move forward and neither will reverse. Both are stuck forever.

12.1 A simple example

Process P1 holds a printer and needs a scanner. Process P2 holds the scanner and needs the printer. Each waits for the other - a deadlock.

```
P1 holds [Printer]  --- wants --->  [Scanner] held by P2
P2 holds [Scanner]  --- wants --->  [Printer] held by P1
```

12.2 Four necessary conditions (Coffman conditions)

Deadlock can occur only if **all four** of these hold at the same time:

Condition	Meaning
Mutual exclusion	At least one resource can be used by only one process at a time.
Hold and wait	A process holds some resources while waiting for others.
No preemption	A resource cannot be forcibly taken away; it must be released voluntarily.
Circular wait	A circle of processes exists where each waits for a resource held by the next.

12.3 Handling deadlocks

Approach	Idea
Prevention	Make sure at least one of the four conditions can never hold. E.g. force a process to request all resources at once (breaks hold-and-wait), or number the resources and request them in order (breaks circular wait).
Avoidance	Check every request and grant it only if the system stays in a *safe state*. The best-known method is the **Banker's Algorithm**.
Detection and Recovery	Allow deadlocks, detect them (using a resource-allocation graph), then recover by terminating processes or taking resources back.
Ignore (Ostrich method)	Pretend deadlocks never happen. Used by most general-purpose systems like Windows and Linux because deadlocks are rare and the other methods are costly.

12.4 Safe state and the Banker's Algorithm

A system is in a **safe state** if there is an order in which all processes can finish even if each asks for its maximum needs. The Banker's Algorithm works like a careful banker: it only gives a loan (resource) if it can still satisfy every customer's maximum demand afterwards. Not every unsafe state is a deadlock, but every deadlock is an unsafe state.

12.5 Starvation vs deadlock

In **starvation** a process waits for a very long time because others always get priority, but the system as a whole is still moving. In a **deadlock**, the processes involved will wait forever with no progress at all.

> **Key Idea**
> Deadlock needs all four conditions. Break any one of them and deadlock cannot happen.

CHAPTER 13

Summary, Practice Questions and Glossary

13.1 Chapter summary

Topic	One-line summary
System software	Programs that manage hardware and support applications, led by the OS.
Process	A program in execution with code, data, heap and stack.
Process states	New, Ready, Running, Waiting, Terminated.
PCB	Data structure storing everything the OS needs to know about a process.
Creation	Parent creates child using fork(); exec() loads a new program.
Scheduling	Decides which ready process uses the CPU; long, short and medium-term schedulers.
Algorithms	FCFS, SJF, Priority, Round Robin, Multilevel Feedback Queue.
Context switch	Saving one process's state and loading another's; costs time.
IPC	Shared memory and message passing let processes cooperate.
Threads	Lightweight units of execution sharing a process's memory.
Synchronization	Mutex, semaphores and monitors prevent race conditions.
Deadlock	Circular waiting for resources; handle by prevention, avoidance, detection or ignoring.

13.2 Practice questions

1. Define a process. How is it different from a program?
2. Draw and explain the process state diagram.
3. What is a PCB? List any six fields stored in it.
4. Explain the roles of the long-term, short-term and medium-term schedulers.
5. Calculate the average waiting time using FCFS for processes with burst times 5, 3 and 8 (all arriving at time 0). *(Answer: 4.33 ms)*
6. What is a context switch? Why is it considered overhead?
7. Compare shared memory and message passing.
8. Differentiate between a process and a thread.
9. What is a race condition? How can semaphores solve it?
10. State the four necessary conditions for deadlock and explain one prevention method.

13.3 Glossary

Term	Meaning
Aging	Gradually increasing the priority of long-waiting processes.
Burst time	CPU time a process needs to complete.
Convoy effect	Short processes stuck behind a long one.
Dispatcher	Module that gives CPU control to the selected process.
Kernel	The core part of the OS with full control over hardware.
Multiprogramming	Keeping several programs in memory at once.
Preemption	Taking the CPU away from a running process.
Race condition	Result depends on the unpredictable order of execution.
Starvation	A process waits indefinitely because others keep going first.
System call	A request from a program to the OS for a service.
Time quantum	Fixed time slice given to a process in Round Robin.